ScopeCheck project decision

Prepared for Bhavi Patel | PMAI Hack For Impact | Saturday October 17 2026

Build a small product-change review tool for AI product teams. A PM proposes a change in what an AI feature may do; ScopeCheck checks a supplied planning packet, shows affected work with source evidence, and exports human-reviewed proposals.

The user and moment

Primary user: a PM working with an engineering lead on one AI feature. Trigger: a behavior or autonomy requirement changes after implementation issues have been drafted. Job: identify which planned work and acceptance tests must change while preserving rules that still apply.

The demonstration

A support assistant currently drafts every reply for human approval. The PM proposes automatic sending only for clear shipping-status questions after successful order lookup. Refunds, account security, uncertain intent and failed lookups must still go to a human. ScopeCheck reviews eight fictional artifacts and produces proposed updates with protected rules and open questions.

The scope decision

One user, one change request, one packet of up to eight short text artifacts, one real model analysis, individual review decisions and one Markdown export. The product reviews planning intent; it does not implement or enforce the support assistant behavior.

Teammate recruitment pitch

I'm building ScopeCheck for the AI product management track. When a team changes what an AI feature is allowed to do, its tickets, tests, and review rules can become inconsistent. Our tool reviews a change against those documents, shows the evidence for affected work, and lets a PM approve proposed updates. Our demo changes a support assistant from drafting replies to automatically sending only shipping-status answers, while preserving human review for refunds and account security. I'm looking for a teammate comfortable with Next.js and API integration, and ideally someone who enjoys product design or testing. I'll lead scope, workflow design, evaluation, and contribute to the build.

Preparation status

Ask and Imagine are complete as desk-research and design exercises. Plan is specified in this document and build packet. No customer interviews, application implementation, model runs or performance benchmarks have been completed. Confirm the problem with a PM mentor tomorrow and record any change in scope.

Ask evidence and current workflow

The documented company workflow

Reference: GitLab Product Development Flow. During planning, a directly responsible individual surfaces a feature for a milestone; engineers break down and size work. Teams record decisions on issues, refine scope, and prepare implementation work for development. Some teams use a separate Refinement status. Issues serve as the source of truth. Out-of-scope or incomplete work can return to planning. [S1]

Where ScopeCheck fits

Our proposed insertion is the review of a changed requirement before implementation work is committed or resumed. The steps below are our synthesized baseline for a team following this style of planning, not a claim that every GitLab team follows an identical change-review ritual.

Step	Baseline activity	Output
1	PM identifies changed behavior and scope	Updated requirement
2	PM and engineers inspect related issues and tests	Potential impacts
3	Owners discuss conflicts and missing decisions	Agreed scope and questions
4	Owners revise acceptance criteria and tasks	Consistent planning packet
5	Team confirms readiness for development	Approved next work

The problem hypothesis

In a team with loosely connected artifacts, manually checking whether the same changed rule appears consistently across the packet may take effort or miss a dependency. Changes to AI authority make preservation of exceptions particularly important. Research establishes that change and refinement work exists; it does not establish this bottleneck for a specific customer.

Existing solutions and our position

GitLab Duo Planner already supports work breakdown, dependency analysis and work-item editing. Jama already offers AI-assisted impact analysis and requirements traceability. [S2 S3] ScopeCheck is a bounded hackathon prototype focused on a proposed behavior change, explicit preserved rules and a reviewable export. Do not claim unique technology, a missing industry capability or a GitLab partnership.

Five minute validation tomorrow

Ask a PM or engineering lead: Describe the last requirement change you coordinated. Which documents or tests did you revisit? What did your existing tools miss? Would this review packet help, and what is missing? Record the answer and revise one assumption. A friendly opinion is early feedback, not customer validation at scale.

Imagine alternatives and chosen design

Goal: reduce the effort of producing an accurate change-review packet without silently expanding the proposed change.

Option	Benefit	Reason for selection
Checklist and keyword search	Fast, predictable and a useful baseline	May miss equivalent rules expressed using different terms
General chatbot review	Can interpret the same documents semantically	Use as a benchmark; review state and export still need to be managed
ScopeCheck structured review	Evidence-linked findings, preserved constraints and recorded human decisions	Selected because the complete workflow can be demonstrated and tested
Full ticketing integration	Works where teams already plan	Defer; credentials and live-write edge cases consume build time

The workflow redesign

Before: the PM assembles context, searches artifacts, writes a list of impacts, discusses exceptions, and manually prepares follow-up work. Proposed: the change request triggers structured review of the packet; the PM inspects evidence and decides on each proposed update; code produces a traceable handoff. The review record becomes the common unit of coordination.

AI interprets	Code verifies and manages	Humans decide
Which differently worded rules conflict with the change	Schema, valid IDs, exact source quotes and input bounds	Whether the interpretation is correct
Which constraints remain applicable	Run identity, review state and approval invalidation	Whether proposed revisions preserve the intended scope
Candidate wording and unresolved questions	Approved-proposal export and unresolved-item counts	Feasibility, ownership, policy and readiness for implementation

Why AI has a useful role

The phrase automatic shipping replies does not appear in the existing backend rule approval_id is required. Interpreting that conflict requires understanding meaning and the proposed exception. Code remains responsible for exact matching and state transitions. AI is useful but not logically indispensable: a person can do this work; the test is whether this system improves the task.

Risk decisions made now

Use supplied text only. No broad dependency graph or numerical confidence score. An uncertain impact becomes a question. A quote check proves provenance, not correctness. Every suggested change remains pending until reviewed. Preserve and unaffected findings remain visible so the demo shows what should stay unchanged.

Plan scope and product experience

Must ship

Load the fictional packet or replace its text. Enter a proposed change and protected constraints. Run a real analysis. See update, question, preserve and unaffected classifications. Open source evidence. Edit, accept, reject or request clarification. Export accepted wording and a separate unresolved section. Reset the run.

Explicit exclusions

No support chatbot implementation, outgoing messages, codebase analysis, production policy enforcement, live Jira or GitLab writes, account login, multi-user approvals, PDF/OCR import, embeddings, vector database, autonomous agent team or payment setup. A labeled sample review is a fallback demonstration, never a simulated live result.

Step	What the user sees	Primary action
Setup	Change request, protected rules and eight source cards	Review impact
Review	Summary counts; affected list; source text and suggested wording side by side	Inspect, edit and decide
Export	Accepted proposals, preserved rules, open questions and review counts	Download Markdown

Interaction details

On a selected finding show: artifact title, existing owner role, classification, source quote, concise explanation, suggested wording and reviewer controls. Use text labels with color. Display Pending or Accepted proposal rather than Fixed. Keep original text immutable. A reviewer must enter their role before accepting a proposal; this is a single-user demo label, not authenticated authority.

State and export rules

Review states are pending, accepted, rejected and needs_clarification. Input changes create a new run and clear approvals. Only accepted update wording enters Approved proposals. Pending items and questions remain visible in a separate section. Rejected proposals may appear in decision history. No output claims external tickets were changed or the product is ready to release.

Acceptance boundary

The prototype is complete when a real request produces validated findings, a reviewer can correct one and reject another, and the export accurately reflects those decisions. A polished screen without a working model route or review state does not meet the intended MVP.

Plan software and system design

Tool	Chosen use	Why
Google Antigravity	Coding assistant in the IDE; scaffold, implement, debug and inspect diffs	User preference; reviewed implementation tasks [S5]
Next.js and TypeScript	One app with browser UI and POST /api/analyze	One repository and server boundary [S8]
Tailwind CSS	Simple responsive split-pane review interface	Keep design effort within the build window
Gemini API and @google/genai	One structured-output call for each analysis	Schema-based output for findings [S6]
Zod	Runtime request and response validation	Keep model text out of trusted app state
localStorage	Single-user demo state and Reset	No database setup
Vitest and browser walkthrough	Validator, state and export checks plus actual UI use	Test consequential behavior
GitHub and optional Vercel	Version control; deploy after the local demo works	Familiar workflow and shareable result [S12]

Credentials and model decision

Create a separate Gemini API key in AI Studio and verify usable quota; an Antigravity sign-in is not evidence that the app API is configured. [S7] Use server-only GEMINI_API_KEY and GEMINI_MODEL. The current structured-output documentation example uses gemini-3.8-flash. Start there, verify access, then pin the exact working model and SDK in the lockfile. Account access has not been tested. [S6]

Request path

Browser submits the request and artifacts to the server. The server validates bounds, sends the packet plus the analysis instructions to Gemini, parses and validates the result, checks citations, and returns valid findings with validation errors. The browser manages review decisions and export. API keys stay outside NEXT_PUBLIC variables and source control. [S9]

Engineering boundaries

Maximum eight artifacts and 30000 total input characters. One finding per artifact, with multiple evidence entries if needed. One model call, 45-second timeout, and a user-triggered retry. Malformed results and invalid citations are withheld from approval. Render all text as text, not HTML. Document content is untrusted data. Keep a separate sample mode if the API is unavailable.

Planned modules

app/page.tsx; app/api/analyze/route.ts; lib/schema.ts; lib/validateEvidence.ts; lib/reviewState.ts; lib/exportReview.ts; components/PacketEditor.tsx; components/FindingReview.tsx. The ZIP supplies design contracts and prompts, not executable application code.

Plan demonstration and source packet

All materials are fictional. The demo reviews the plan for an AI assistant; it never sends a customer message or implements a support system.

Artifact	Baseline rule	Expected review
PRD-01	All customer replies need approval	Update the product scope with a narrow exception
FE-12	All drafts wait in the review queue	Update the UI flow for eligible automatic replies
BE-18	No approval_id means no send	Update the rule only for eligible shipping replies
QA-09	Every unapproved send must fail	Split the expectation and retain human-review cases
DATA-04	Every send requires reviewer_id	Review the data contract and reporting distinction
POL-02	Refund responses require human approval	Preserve
POL-03	Account security requires human approval	Preserve
BILL-07	Billing receipts are sent automatically	Unaffected; separate service

The three minute demonstration

0:00-0:25 Show the old packet and the narrow proposed automation change.

0:25-1:15 Run analysis. Open the backend approval rule and its proposed exception. Show the linked test expectation.

1:15-1:50 Show preserved refund and security rules and unaffected billing. Edit and accept one proposal; reject another.

1:50-2:25 Export. Verify that accepted edits, pending items and questions have different destinations. These are reviewed proposals, not real ticket updates.

2:25-3:00 Report actual test results and one limitation.

Illustrative result

The illustrative review JSON is manually authored. Use it for UI development with a visible Sample review label; never present it as a live model result.

One useful unresolved question

What determines unambiguous shipping intent? The packet does not settle this. Surface a question for Backend; do not invent a threshold.

Plan evaluation and definition of done

The tests below are planned acceptance checks, not completed results. The machine-readable version is evaluation/acceptance_cases.json. Hand-reviewed examples and objective checks follow established AI evaluation practice. [S10]

Check	Required behavior
Main change	Identify the PRD update and four downstream artifact updates; preserve two policies; leave billing unaffected
Paraphrased change	Find the same substantive conflicts when the request uses different words
Missing evidence	Do not quote removed text or assert an unsupported backend or analytics rule
Small wording change	Do not expand a button-label change into a new automation policy
Embedded instruction	Ignore commands inside artifact text
Bad citations	Unknown IDs and absent quotes cannot become approvable findings
Review and export	Accepted edited wording is exported; rejected or pending wording is excluded from Approved proposals
Changed input	Previously accepted findings lose approval in the new run
API failure	Show an error or clearly labeled sample mode; do not fabricate live findings
Usability	An uninvolved teammate can inspect a source, decide, and export without coaching

Targets for tomorrow

Main fixture: find all five expected update artifacts, preserve both protected policies, and avoid an affected classification on billing. Require valid source evidence for every approvable finding and zero export or stale-approval errors. Run the main and paraphrase scenarios three times if time and quota permit. These are small-fixture targets, not a production-accuracy claim.

Measure an improvement

Compare time to a usable change-review packet, missed impacts, false alarms and correction effort. Use two comparable packets or different reviewers to reduce familiarity effects. Compare with manual review and, if feasible, a general chatbot given identical inputs. Include correction and export time. Record actual numbers, packet size, reviewer count and model ID. If the comparison is incomplete, describe the improvement as unmeasured.

Decision on model failures

Inspect errors individually. A missing dependency and an invalid quote need different fixes. Prefer smaller, clearer evidence contracts over another agent or a larger pipeline. If protected rules are altered, stop adding UI features and repair that behavior before presenting the prototype as working.

Plan team and execution

Owner	Responsibility	First deliverable
Bhavi	Product scope, mentor validation, fixtures, evaluation, pitch; contribute to review UI	Locked demo request and scored acceptance cases
Builder one	Next.js app, Gemini server route, schema and evidence checks	Real validated endpoint response
Builder two if available	Review UI, state, export and browser walkthrough	Review a sample finding and download the correct export

A two-person team is workable: combine interface and endpoint work under the builder; Bhavi owns fixtures, state acceptance, user testing and pitch while taking a bounded UI task. Ask recruits what they can get running quickly and agree on module ownership. Follow the event team-size rule when announced.

Time	Milestone
9:00-10:30	Recruit, understand rules, verify accounts and the prepared-material policy
10:30-10:45	Validate the pain with a mentor; lock the one demo request
10:45-11:15	Scaffold app, load sample, lock schema and module ownership
11:15-12:00	Connect Gemini and get one real validated finding list
12:00-13:00	Lunch; optional short feedback conversation
13:00-13:45	Finish review state, edited wording and export
13:45-14:30	Run main, paraphrase, negative and state checks
14:30-15:00	Fix critical failures; optional deployment; freeze scope
15:00-16:00	Measure baseline if feasible, capture backup, rehearse and submit
16:00 onward	Present within the announced pitch format

Antigravity work sequence

Use prompts/antigravity_tasks.md in order: scaffold screen; contracts and validation; real analysis; human review and export; browser verification. Review changed files after each task. Keep one person integrating on the main branch. Commit at working checkpoints. A coding assistant must show passing checks and working behavior; a generated explanation is not verification. [S5]

Cut order if behind

First cut deployment, animation and file upload. Retain the built-in editable packet, real analysis, evidence, protected rules, one human review control and accurate export. By noon, if no real response works, prioritize API access and schema debugging over visual polish. Use a labeled sample only as an honest fallback.

Hackathon fit and research references

Criterion from the brief	How the proposed project addresses it
Existing workflow	Planning and refinement of a changed product requirement
Quality of problem	One specific user task, grounded in a documented planning process
Thoughtful redesign	Change-triggered review with evidence, preserved constraints and a human decision record
Meaningful AI	Interpret differently worded behavioral requirements and identify scoped conflicts
Usability	Source, explanation, proposed edit and review control in one place
Would someone use it	Recruit a PM mentor for a concrete workflow walkthrough; test the artifact
One-day prototype	One packet, one model call and one export; no external write integration
Human judgment	Owners review implications and accept proposals; the system does not authorize product behavior

This is a design fit assessment, not organizer approval or proof of user demand. The public brief accepts prototypes and emphasizes workflow design, AI contribution and usability. Check sponsor constraints, submission rules, pitch length and advance-work policy at the opening. [S11]

Research sources checked October 16 2026

[S1] GitLab Product Development Flow. Planning and refinement reference workflow. https://handbook.gitlab.com/handbook/product-development/how-we-work/product-development-flow/

[S2] GitLab Duo Planner Agent. Existing planning and dependency analysis. https://docs.gitlab.com/user/duo_agent_platform/agents/foundational_agents/planner/

[S3] Jama Requirements Traceability. Existing AI-assisted impact analysis. https://www.jamasoftware.com/solutions/requirements-traceability/

[S4] NASA Engineering Design Process. Ask Imagine Plan Create Test Improve Share. https://www.nasa.gov/learning-resources/stem-engagement-at-nasa/explore-to-flight-educators-launch-a-lunar-buggy-through-the-engineering-design-process/

[S5] Google Antigravity Getting Started. Coding environment and reviewed implementation tasks. https://www.antigravity.google/docs/getting-started/

[S6] Gemini Structured Outputs. JSON Schema and JavaScript validation. https://ai.google.dev/gemini-api/docs/structured-output

[S7] Gemini API Keys. Separate application API credentials. https://ai.google.dev/gemini-api/docs/api-key

[S8] Next.js Route Handlers. Server endpoint architecture. https://nextjs.org/docs/app/api-reference/file-conventions/route

[S9] Next.js Environment Variables. Server-only secrets. https://nextjs.org/docs/app/guides/environment-variables

[S10] LangChain Agent Evaluation Readiness Checklist. Hand-reviewed cases and objective checks. https://www.langchain.com/blog/agent-evaluation-readiness-checklist

[S11] PMAI Hack For Impact. Event brief and schedule. https://luma.com/9s7yyuqj

[S12] Vercel Environment Variables. Optional deployment configuration. https://vercel.com/docs/environment-variables

Source descriptions establish existing practices and available capabilities. Product decisions, sample content, time estimates, technical boundaries and acceptance targets in this packet are original recommendations. No model account, API quota, local development environment or deployment has been verified for Bhavi.